

RPG Combat System Implementácia

FIIT STU

DOMINIK JURKAS

Obsah

Úvod	2
Zmeny voči zadaniu 2	2
Views	3
Funkcie.....	4
Indexovanie	13
Fyzický model	14
Používateľská príručka	15
1. Vytvorenie tabuliek	15
2. Vloženie údajov do tabuliek.....	15
3. Nastavenie views a indexov	15
4. Vytvorenie funkcií a procedúr	16
5. Vyčistenie databázy (voliteľné)	16
Príklad postupu pri spustení skriptov.....	16
Testovanie	17
Test #1: Založenie prvého combatu, vytvorenie prvého kola a pridanie hráčov 1 a 2..	17
Test #2: Hráč 1 zaútočí na hráča 2 pomocou Iron Sword v kole 1	17
Test #3: Hráč 2 zaútočí na hráča 1 pomocou Fireball.....	18
Test #4: Hráč 1 sa vylieči použitím Healing Potion	18
Test #5: Hráč 2 vyhodí Healing Potion na bojisko, hráč 1 ho následne zoberie	18
Test #6: Hráč 1 sa pokúsi použiť item na ktorý nemá dostatok AP.....	19
Test #7: Hráč 2 zabije hráča 1 Fireballom	19
Test #8: Reset kola – nové kolo vytvorené, AP pre hráča 2 resetované	19
Test #9: Pokus hráča 2 zobrať príliš ťažký item pre jeho inventár	20
Test #10: Pokus hráča 2 zaútočiť na hráča 3, ktorý nie je v aktívnom combate	20
Test #11: Pokus hráča 2 sa pridať do aktívneho kola, ale hráč 2 už je v aktívnom kole	20
Test #12: Vytvorenie nového combatu, pridanie hráča 1 do combatu, kontrola či sa zregeneroval kým nebol v boji	21
Test #13: Ukončenie combatu, premiestnenie hráča 2 preč z aktívneho combatu	21
Test Views:	21
Záver	22
Použité nástroje	22

Úvod

V tomto treťom zadaní implementujeme návrh databázy z druhého zadania. Cieľom tohto zadania je vytvoriť databázové štruktúry a funkcionality podľa predchádzajúceho návrhu, pričom sa zameriame na implementáciu funkcií, views, indexov a samozrejme na testy, ktoré ukážu ich správnosť a funkčnosť v praxi.

Zmeny voči zadaniu 2

Upravil som vzťah medzi User a Character. Predtým bol User spojený s Character cez character_id, teraz je to však tak, že Character obsahuje user_id, čím sa upravil tento vzťah.

Ďalšou zmenou, ktorú som implementoval, je pridanie základného AP cost pre použitie predmetov v tabuľke Items a pre použitie kúzli v tabuľke Spells. Tento atribút je teraz prítomný, aby bolo možné správne určiť náklady na použitie predmetu alebo kúzla.

Napokon som pridal nový atribút round_order do tabuľky Rounds, ktorý zabezpečuje správnu identifikáciu kôl v prípade viacero súbežných bojov. Táto zmena je nevyhnutná, pretože v prípade viacerých súbežných bojov môže dôjsť k situácii, kedy by kolo v boji č. X začínalo s ID 10, namiesto očakávaného ID 1. round_order teda zabezpečuje správnosť identifikácie kôl v rámci daného boja, pričom každý round má svoj vlastný identifikátor pk id.

Views

1. v_combat_state

Tento view získava stav všetkých hráčov pripojených k aktívnemu boju a ich action point stav v danom kole. Pre každý aktívny boj sa vypočíta, koľko AP hráč už minul v tomto kole a koľko mu ešte zostáva. Čiže zisťuje, ktorí hráči sú pripojení k aktuálnemu boju a ešte neodišli ('JOIN' v Player_combat_log), spočítava minuté AP, ktoré už hráči použili v rámci aktuálneho kola (z Round_events).

2. v_character_full_stats

Tento view vykonáva výpočty pre každý atribút postavy a zabezpečuje, že sa zobrazí najnovšia hodnota pre každý atribút, vrátane jej základných atribútov (Strength, Dexterity, Constitution, Intelligence), maximálneho zdravia (MaxHealth), aktuálneho zdravia (Health), maximálneho počtu AP (maxAP), váhového limitu inventára (max_inventory_weight), aktuálnej váhy inventáru, spočítava Armor Class (AC) z Dexterity a bonusov z Classes.

3. v_most_damage

Tento view sumarizuje celkové poškodenie spôsobené všetkými hráčmi (ktorí použili úspešný útok) počas boja. Výsledky sú zoradené podľa celkového spôsobeného poškodenia.

4. v_strongest_characters

Tento view zobrazuje najsilnejších hráčov na základe dvoch kritérií: celkového poškodenia, ktoré spôsobili, a aktuálneho zdravia. Zoradenie je podľa celkového poškodenia, pričom ak majú hráči rovnaké poškodenie, ďalej sa zoradí podľa zostávajúceho zdravia.

5. v_combat_damage

Tento view sumarizuje celkové poškodenie spôsobené počas každého boja. Pre každý combat_id zobrazuje sumu poškodenia, ktorá bola spôsobená počas všetkých kôl v rámci daného boja.

6. v_spell_statistics

Tento view zobrazuje štatistiky o kúzlach, ktoré boli použité počas boja. Zahŕňa počet použitia kúzla, počet úspešných zásahov, počet neúspešných zásahov, celkové spôsobené poškodenie a priemerné poškodenie na úspešný zásah.

Funkcie

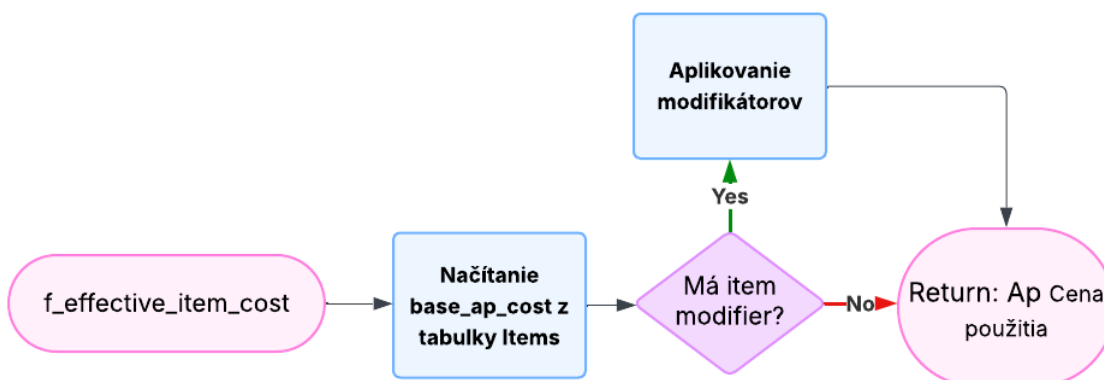
1. `f_calculate_attack_roll(p_attacker_id bigint, p_method text)`

- **Čo robí:** Vypočíta hodnotu útoku na základe metódy útoku ('weapon' alebo 'spell'). Pre zbraň sa použije **Strength/2**, pre kúzla **Intelligence/2** a pridá sa hodnota hodu kockou (d20).
- **Čo kontroluje:** Kontroluje, aký spôsob útoku (zbraň alebo kúzlo) je použitý a na základe toho vypočíta príslušný bonus (Strength alebo Intelligence).



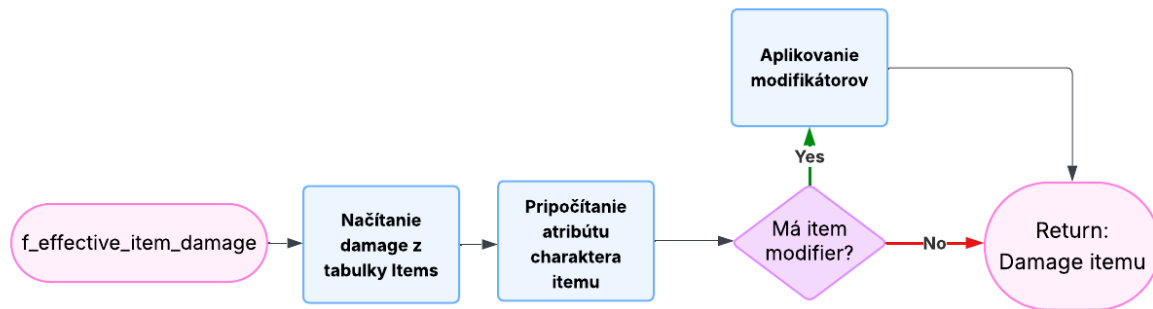
2. `f_effective_item_cost(p_item_id bigint, p_character_id bigint)`

- **Čo robí:** Vypočíta efektívnu cenu AP pre použitie predmetu, berie do úvahy základnú cenu predmetu a modifikátory.
- **Čo kontroluje:** Kontroluje, či existujú nejaké modifikátory pre daný predmet, ktoré ovplyvňujú jeho cenu.



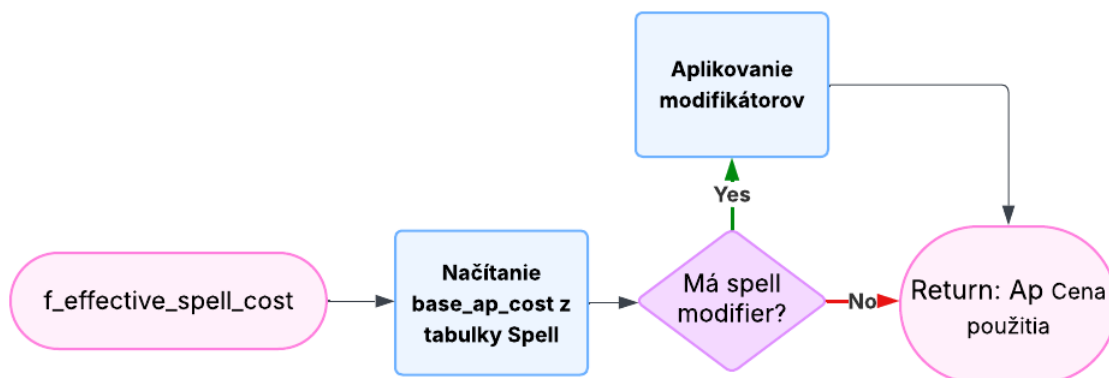
3. f_effective_item_damage(p_item_id bigint, p_user_id bigint)

- **Čo robí:** Vypočíta efektívne poškodenie spôsobené predmetom na základe jeho základného poškodenia, atribútov postavy a príslušného modifikátora.
- **Čo kontroluje:** Kontroluje, ktoré atribúty postavy ovplyvňujú poškodenie spôsobené predmetom, ako aj prítomnosť akýchkoľvek modifikátorov na predmet.



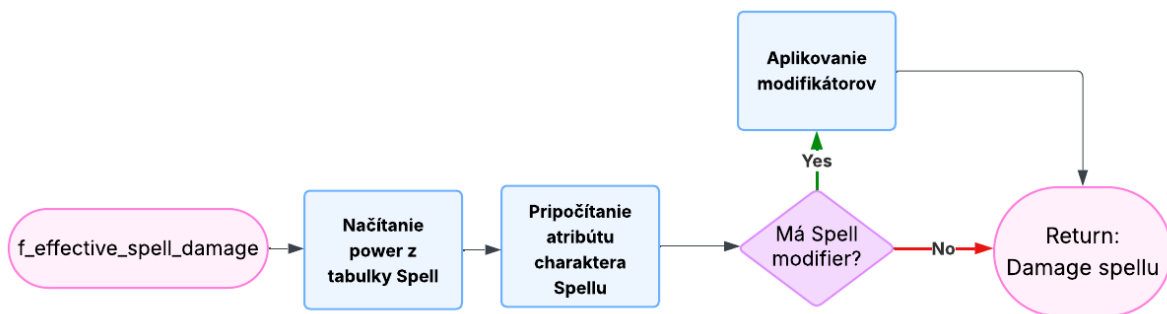
4. f_effective_spell_cost(p_spell_id bigint, p_caster_id bigint)

- **Čo robí:** Vypočíta efektívnu cenu AP pre kúzlo na základe jeho základnej ceny a prípadných modifikátorov založených na atribútoch postavy.
- **Čo kontroluje:** Kontroluje, či existujú nejaké modifikátory pre dané kúzlo, ktoré ovplyvňujú jeho cenu (napr. modifikátory z atribútov postavy).



5. f_effective_spell_damage(p_spell_id bigint, p_caster_id bigint)

- **Čo robí:** Vypočíta efektívne poškodenie spôsobené kúzlom, ktoré berie do úvahy jeho základné poškodenie a príslušné modifikátory.
- **Čo kontroluje:** Kontroluje modifikátory založené na atribútoch postavy a prítomnosť akýchkoľvek špecifických modifikátorov kúzla.



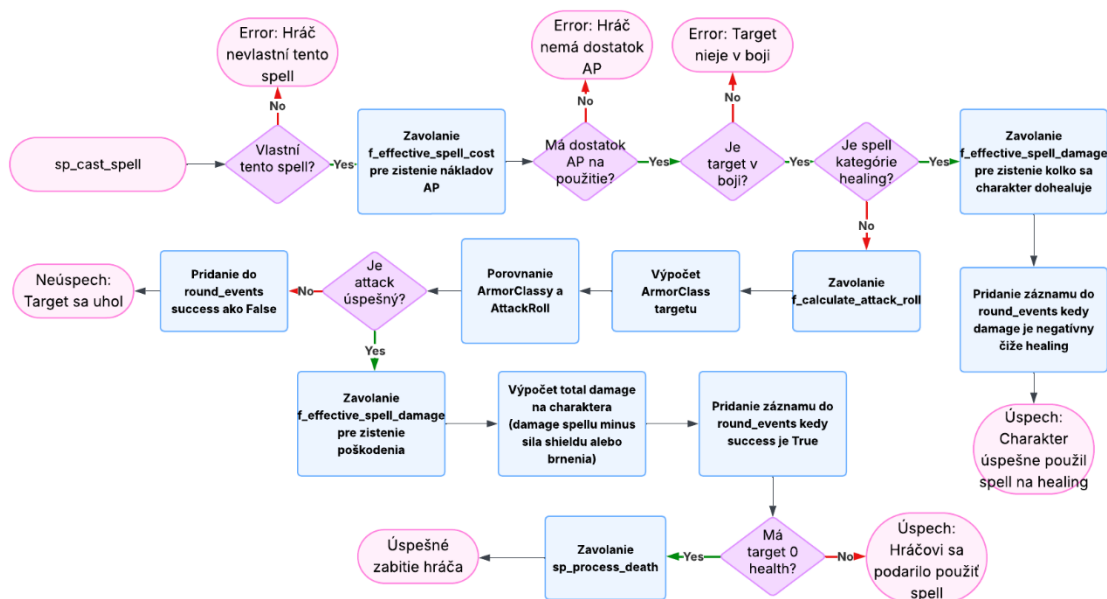
6. f_get_round_status(p_round_id bigint)

- **Čo robí:** Získava informácie o aktuálnom stave kola, ako sú maximálne AP, minulé AP a zostávajúce AP pre každého hráča v danom kole.
- **Čo kontroluje:** Kontroluje minulé AP v danom kole a porovnáva s maximálnym počtom AP hráčov v kole.



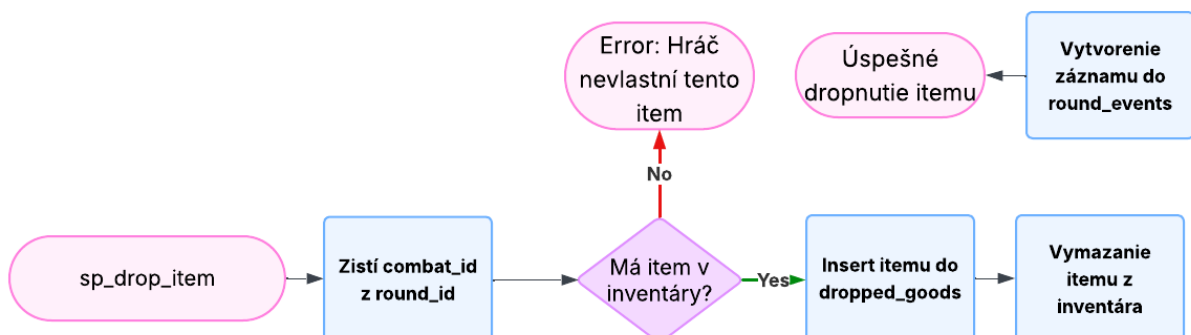
7. sp_cast_spell(p_combat_id bigint, p_round_id bigint, p_caster_id bigint, p_target_id bigint, p_spell_id bigint, p_wand_id bigint DEFAULT NULL)

- **Čo robí:** Vykoná kúzlo, ktoré je použitá na cieľ. Vypočíta poškodenie spôsobené kúzlom a aplikuje ho na cieľ. Ak cieľ zabije, zavolá funkcie ktoré procesnú túto smrť.
- **Čo kontroluje:** Kontroluje, či má character dostatočné AP na vykonanie kúzla, či je cieľ v boji , či kúzlo zasiahne cieľ (kontrola útoku), či character vlastní daný spell , skontroluje či po použití útoku je život cieľa 0.



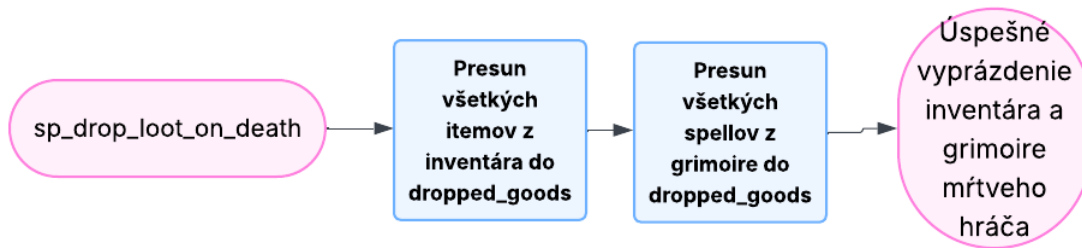
8. sp_drop_item(p_round_id bigint, p_character_id bigint, p_item_id bigint)

- **Čo robí:** Zaznamená položku, ktorú postava dropne počas kola, a presunie ju do boja (do tabuľky Dropped_goods).
- **Čo kontroluje:** Kontroluje, či postava vlastní predmet, ktorý sa pokúša dropnúť, a či je predmet dostupný na bojisku.



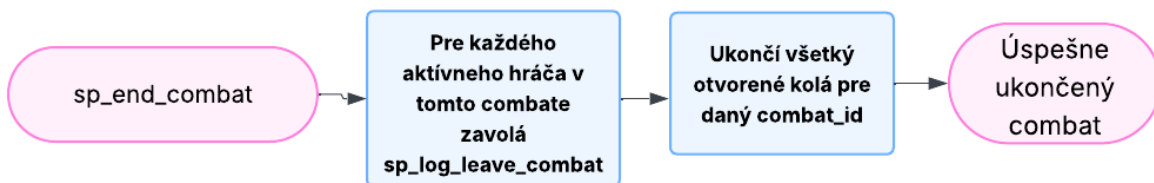
9. sp_drop_loot_on_death(p_combat_id bigint, p_character_id bigint)

- **Čo robí:** Presunie všetky predmety a kúzla postavy, ktorá zomrela, na bojisko a vyprázdni jej inventár.
- **Čo kontroluje:** Kontroluje, či postava má nejaké predmety alebo kúzla, ktoré môžu byť presunuté na bojisko.



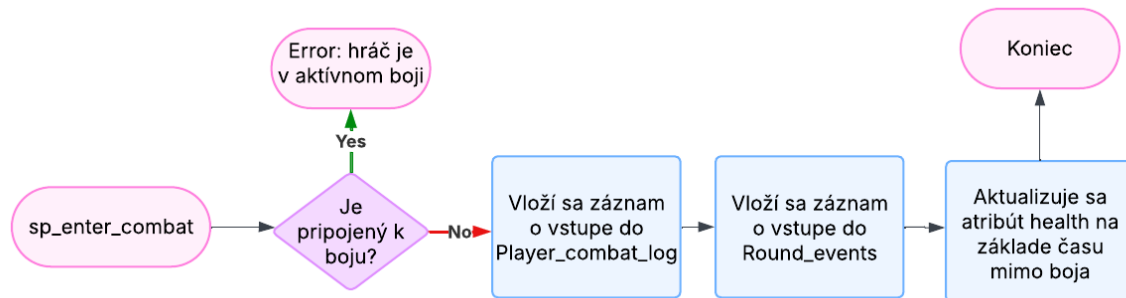
10. sp_end_combat(p_combat_id bigint)

- **Čo robí:** Ukončí všetky otvorené kolá pre daný `combat_id` a zaznamená finálny event v poslednom kole.
- **Čo kontroluje:** Skontroluje, či existujú otvorené kolá pre daný `combat`, ktoré je potrebné uzavrieť.



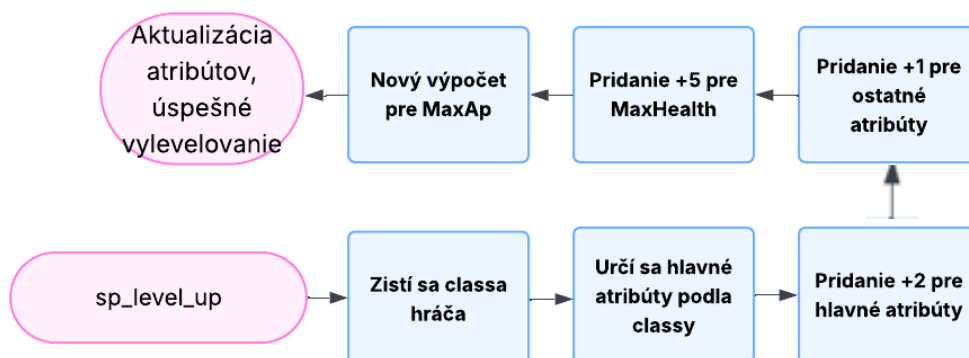
11. sp_enter_combat(p_combat_id bigint, p_character_id bigint)

- **Čo robí:** Pridá hráča do boja (combat) a zabezpečí jeho regeneráciu HP pred vstupom.
- **Čo kontroluje:** Kontroluje, či je hráč už v boji, a ak nie, pridá ho do boja. Kontroluje tiež, či je potrebné regenerovať HP na základe času, ktorý prešiel od jeho posledného odchodu.



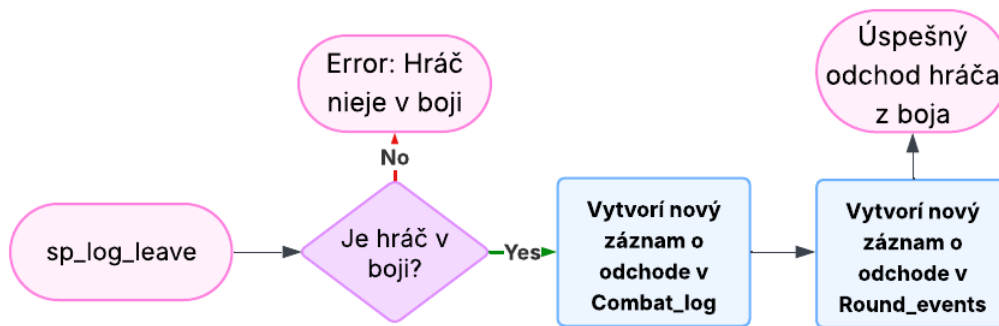
12. sp_level_up(p_winner_id bigint)

- **Čo robí:** Vykoná level up pre postavu, ktorá niekoho zabila, a zvyšuje jej atribúty (strength, dexterity, etc.) a maximálne HP.
- **Čo kontroluje:** Skontroluje triedu víťaza a na základe nej zvyšuje relevantné atribúty.



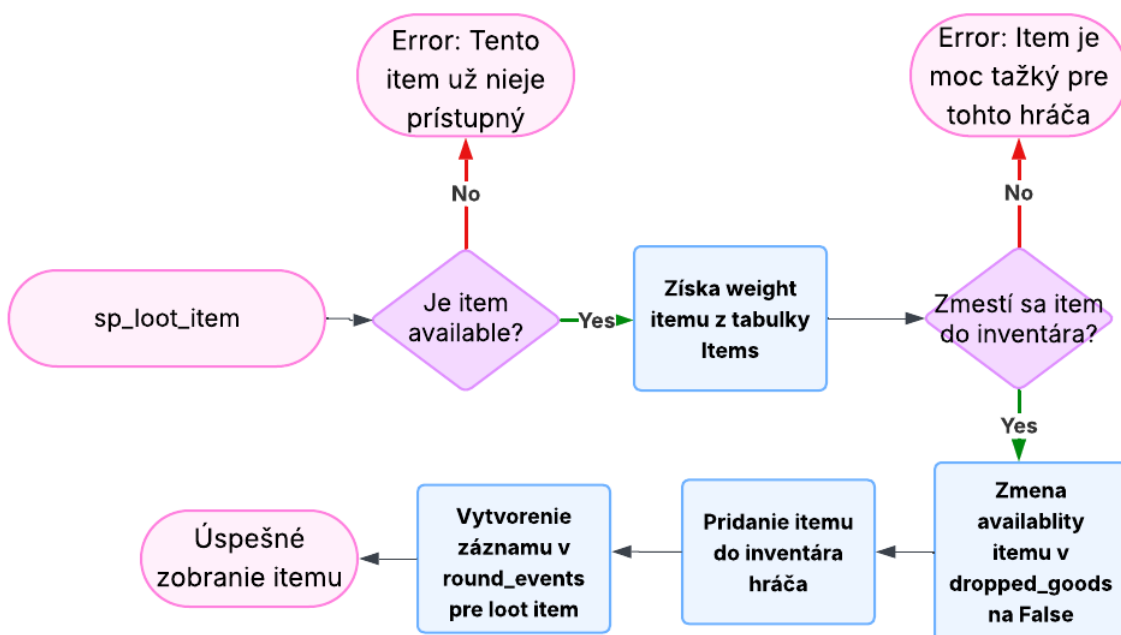
13. sp_log_leave_combat(p_combat_id bigint, p_character_id bigint)

- **Čo robí:** Zaznamená odchod hráča z boja a zapisuje tento event do Player_combat_log a Round_events.
- **Čo kontroluje:** Skontroluje, či je postava ešte v boji pred zaznamenaním jej odchodu.



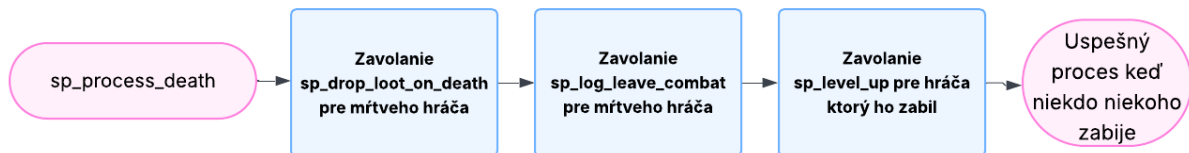
14. sp_loot_item(p_combat_id bigint, p_round_id bigint, p_character_id bigint, p_item_id bigint)

- **Čo robí:** Umožní postave vziať predmet z bojiska, ak splní podmienky (napr. váhový limit).
- **Čo kontroluje:** Kontroluje, či je predmet dostupný na bojisku a či postava nemá prekročený váhový limit v inventári.



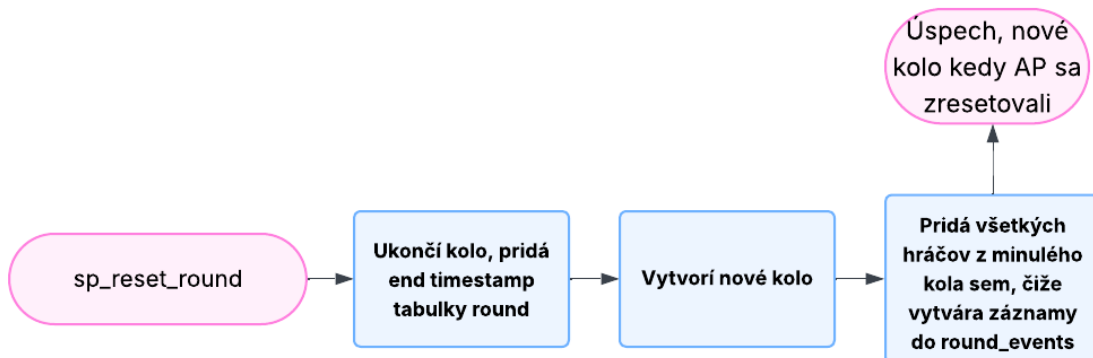
15. sp_process_death(p_combat_id bigint, p_dead_id bigint, p_killer_id bigint)

- **Čo robí:** Spracuje smrť postavy, presunie jej loot na bojisko, odstráni ju z boja a vykoná level-up pre zabijaka.
- **Čo kontroluje:** Kontroluje, či postava, ktorá zomrela, má nejaký loot, ktorý môže byť presunutý na bojisko.



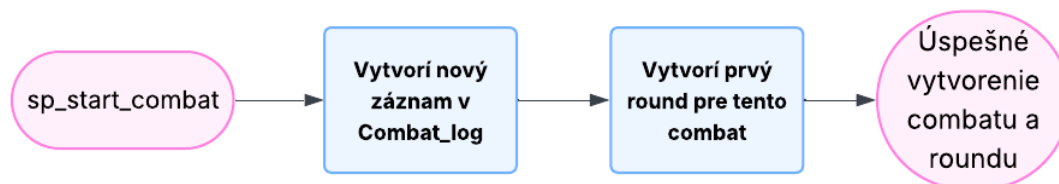
16. sp_reset_round(p_combat_id bigint)

- **Čo robí:** Ukončí aktuálne kolo a vytvorí nové kolo v danom boji, presunie tam všetkých živých charakterov, ktorý bojovali v tomto kole.
- **Čo kontroluje:** Skontroluje, či je aktuálne kolo ukončené a vytvorí nové kolo s aktualizovanými hráčmi.



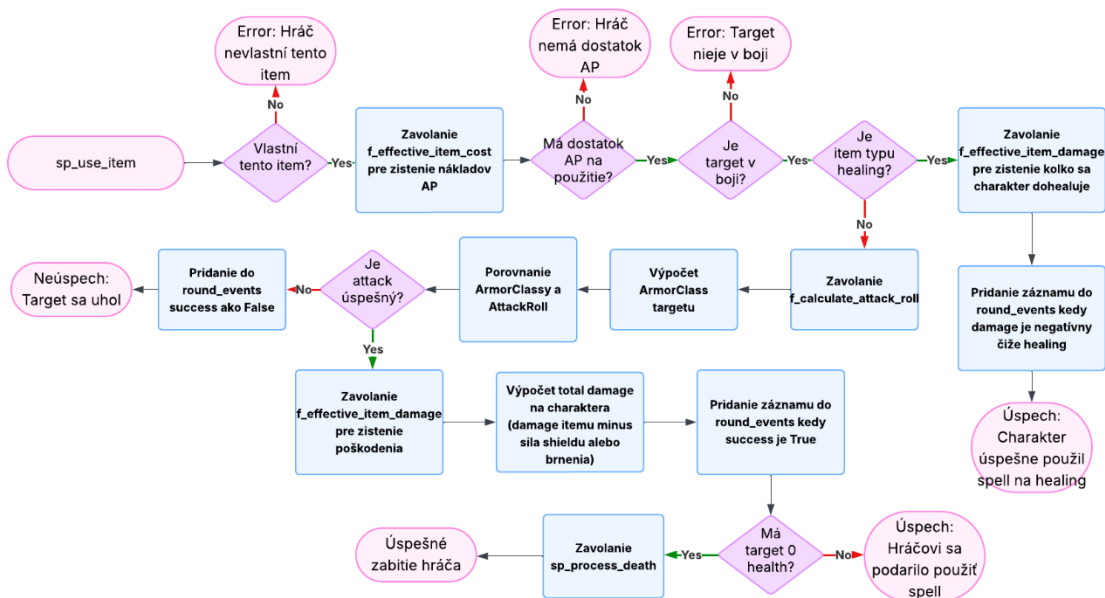
17. sp_start_combat()

- **Čo robí:** Začína nový boj a vytvára prvé kolo.
- **Čo kontroluje:** Kontroluje, či je boj správne nastavený a či je vytvorené prvé kolo pre daný combat.



18. sp_use_item(p_combat_id bigint, p_round_id bigint, p_user_id bigint, p_target_id bigint, p_item_id bigint)

- **Čo robí:** Umožňuje postave použiť predmet na seba alebo na cieľ (napr. liečenie alebo útok), ak cieľ zabije, zavolá funkcie ktoré procesnú túto smrť.
- **Čo kontroluje:** Kontroluje, či má character dostatočné AP na vykonanie útoku, či je cieľ v boji , či zasiahne cieľ (kontrola útoku), či character vlastní daný item , skontroluje či po použití útoku je život cieľa 0.



Indexovanie

Popis jednotlivých indexov:

1) `idx_round_events_round_id`

Tento index je vytvorený na stĺpci `round_id` v tabuľke `round_events`, čo umožňuje rýchle vyhľadávanie všetkých udalostí (events) pre konkrétne kolo. Využíva sa to, keď chceme rýchlo načítať všetky udalosti, ktoré sa odohrali v určitom kole.

2) `idx_round_events_player2_success`

Tento index je vytvorený na kombinovaných stĺpcoch `player2_id` a `success` v tabuľke `round_events`. Pomáha optimalizovať vyhľadávanie v prípade, že potrebujeme rýchlo získať súčet alebo počet úspešných poškodení (damage) podľa konkrétneho cieľa (`player2_id`).

3) `idx_character_attributes_char_attr`

Tento index je vytvorený na kombinovaných stĺpcoch `character_id` a `attribute_id` v tabuľke `character_attributes`. Umožňuje rýchly prístup ku konkrétnym atribútom postavy (napríklad na získanie hodnoty atribútu ako Strength alebo Dexterity pre konkrétnu postavu).

4) `idx_player_combat_log_combat_id`

Tento index je vytvorený na stĺpci `combat_id` v tabuľke `player_combat_log`. Tento index je užitočný pre rýchle vyhľadávanie všetkých hráčov pripojených k určitej bojovej relácii (combat session).

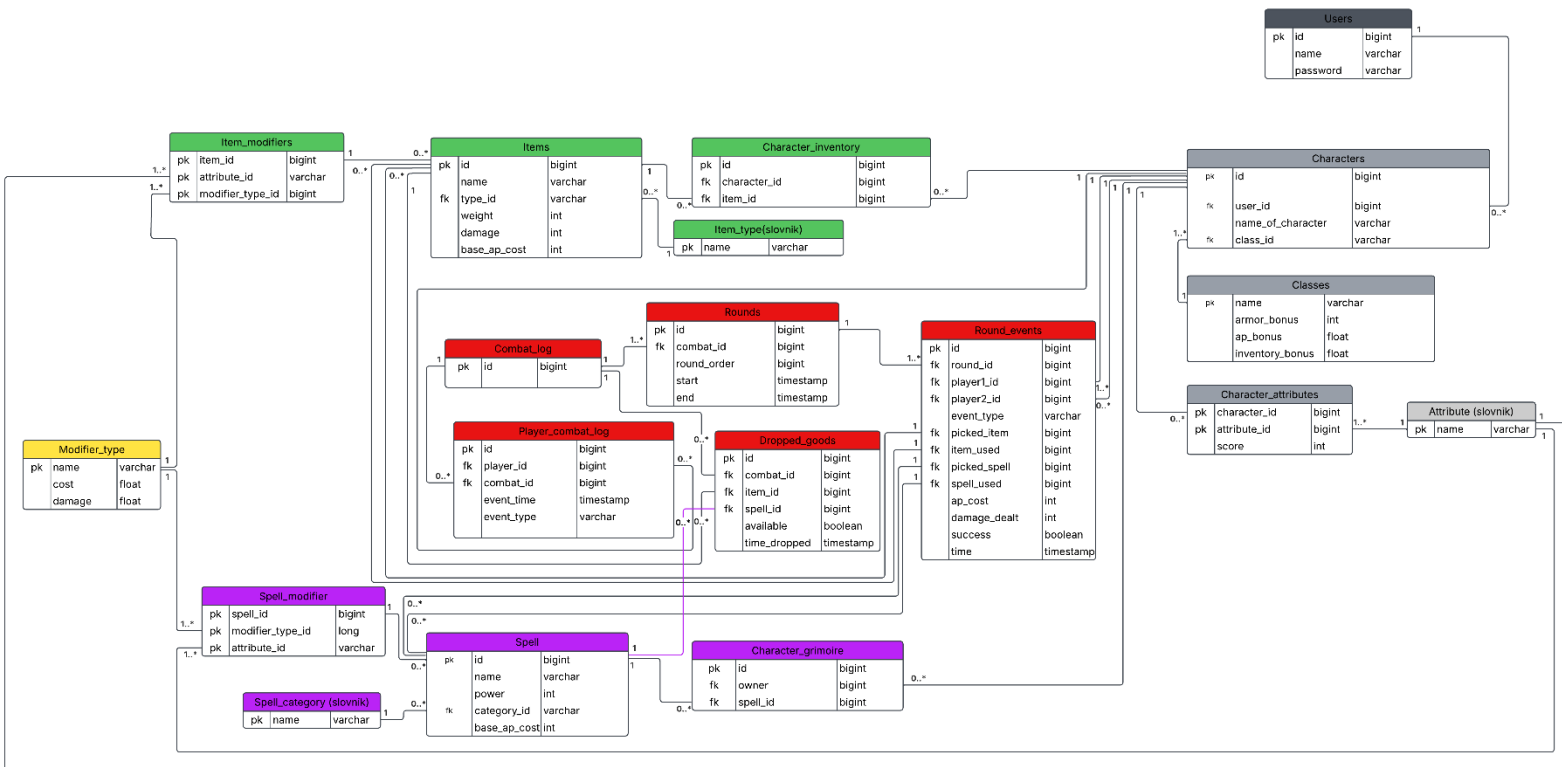
5) `idx_rounds_combat_id`

Tento index je vytvorený na stĺpci `combat_id` v tabuľke `rounds`. Umožňuje rýchle vyhľadávanie všetkých kôl v rámci danej bojovej relácie (combat).

Zhrnutie:

Tieto indexy som navrhol na optimalizáciu výkonu dotazov v databáze. Každý z nich je zameraný na časté operácie a dotazy, ktoré sa používajú na vyhľadávanie konkrétnych dát v systéme, ako sú eventy, postavy, boje a atribúty. Použitie indexov je kľúčové pre udržanie výkonu systému pri práci s veľkými objemami dát, čím sa zabezpečuje, že operácie ako vyhľadávanie, filtrovanie budú prebiehať čo najrýchlejšie.

Fyzický model



Používateľská príručka

1. Vytvorenie tabuliek

Na začiatku je potrebné spustiť SQL skript **create_alltables.sql**, ktorý vytvorí všetky potrebné tabuľky v databáze.

Kroky:

- Otvorte databázový nástroj (napr. pgAdmin)
- Pripojte sa k vašej databáze.
- Spustite SQL skript **create_alltables.sql**, ktorý obsahuje príkazy na vytvorenie všetkých tabuliek.

Tento skript obsahuje príkazy na vytvorenie tabuliek, ktoré sú potrebné na správnu funkčnosť systému.

2. Vloženie údajov do tabuliek

Po vytvorení tabuliek je potrebné vložiť niektoré testovacie údaje do týchto tabuliek. Na tento účel použijeme SQL skript **insertUdajov.sql**.

Kroky:

- Spustite SQL skript **insertUdajov.sql**, ktorý obsahuje príkazy na vloženie preddefinovaných údajov do tabuliek.

Tento skript vloží potrebné údaje ako napríklad postavy, predmety, kúzla a iné základné dáta potrebné pre testovanie a fungovanie systému.

3. Nastavenie views a indexov

Ďalej je potrebné spustiť skript **ViewsAndIndexes.sql**, ktorý vytvorí všetky pohľady (views) a indexy na optimalizáciu výkonu dopytov.

Kroky:

- Spustite SQL skript **ViewsAndIndexes.sql**, ktorý obsahuje príkazy na vytvorenie pohľadov a indexov.

Tento skript vám umožní efektívne pristupovať k požiadavkám na zobrazenie dát a zabezpečiť rýchle vyhľadávanie v databáze.

4. Vytvorenie funkcií a procedúr

Posledným krokom je spustenie skriptu **funkcie.sql**, ktorý vytvorí všetky funkcie a procedúry potrebné na správne fungovanie systému.

Kroky:

- a) Spustíte SQL skript **funkcie.sql**, ktorý obsahuje príkazy na vytvorenie všetkých funkcií a procedúr.

Tento skript zabezpečí, že všetky funkcie (napr. pre začatie boja, použitie predmetov, útoky, resetovanie kôl, atď.) budú správne implementované a pripravené na použitie.

5. Vyčistenie databázy (voliteľné)

Ak by ste potrebovali vyčistiť databázu a vymazať všetky údaje z tabuliek, môžete použiť skript **cleanup.sql**.

Kroky:

- a) Spustíte SQL skript **cleanup.sql**, ktorý vykoná nasledujúce operácie:

Vymaže všetky údaje z tabuliek. Resetuje identitu (primárne kľúče budú nastavené na začiatok). Tento skript môže byť užitočný pri teste alebo ak potrebujete začať znovu s čistou databázou.

Príklad postupu pri spustení skriptov

1. Spustíte **create_alltables.sql**.
2. Spustíte **insertUdajov.sql** na vloženie testovacích údajov.
3. Spustíte **ViewsAndIndexes.sql** pre vytvorenie pohľadov a indexov.
4. Spustíte **funkcie.sql** na vytvorenie funkcií.
5. Ak potrebujete vyčistiť databázu, spustíte **cleanup.sql**.

Testovanie

Pre testovanie treba otvoriť skript **testing.sql**. Tento skript sa nemá spúšťať ako celok, ale postupne po transakciách. Navod a očakavany výstup je v komentároch skriptu. Ak náhodou sa nevydarí útok pri nejakých testoch, tak to treba opakovať, ak náhodou nebude mať character dostatok AP, a není to test ktorý toto kontroluje, tak treba vymazať záznam z round_events kde bol attack neúspešný, tým že sa AP počíta dynamicky, toto zaručí že na ďalší útok bude mať dostatok AP.

Test #1: Založenie prvého combatu, vytvorenie prvého kola a pridanie hráčov 1 a 2

Tento test overuje správnosť inicializácie boja (combat) a pridania hráčov do daného boja.

Funkcia sp_start_combat() vytvorí nový combat záznam a vráti ID boja.

Funkcie sp_enter_combat(1,1) a sp_enter_combat(1,2) pripoja hráčov s ID 1 a 2 do tohto boja.

Očakávaný výstup:

V tabuľke Combat_log sa vytvorí jeden záznam s id = 1.

V tabuľke Rounds sa vytvorí jeden záznam s combat_id = 1.

V tabuľke Player_combat_log sa objaví záznam pre oboch hráčov.

Test #2: Hráč 1 zaútočí na hráča 2 pomocou Iron Sword v kole 1

Tento test kontroluje použitie predmetu (Iron Sword) ako útok na hráča 2 v kole 1.

Funkcia sp_use_item() vykonáva útok.

Očakávaný výstup:

Záznam v Round_events ukáže úspešný útok/neúspešný.

Kontrola, či sa damage správne aplikuje na Health a MaxHealth hráča 2.

Overenie, že damage bolo úspešne aplikované v Round_events.

Test #3: Hráč 2 zaútočí na hráča 1 pomocou Fireball

Tento test overuje použitie kúzla Fireball hráča 2 proti hráčovi 1.

Funkcia `sp_cast_spell()` spustí útok.

Očakávaný výstup:

Záznam v `Round_events` ukáže útok, ktorý bol vykonaný pomocou kúzla.

Skontrolujeme, či sa damage správne aplikuje na hráča 1.

Overenie aplikácie damage na Health a MaxHealth hráča 1.

Test #4: Hráč 1 sa vylieči použitím Healing Potion

Tento test kontroluje, či hráč 1 správne použije liečivý predmet Healing Potion na seba.

Funkcia `sp_use_item()` sa použije na liečenie.

Očakávaný výstup:

Záznam v `Round_events`, kde je použité attack sám na seba.

Overenie, že HP hráča 1 sa správne zvýšilo.

Skontrolovanie, že predmet Healing Potion sa odstránil z inventára hráča 1.

Test #5: Hráč 2 vyhodí Healing Potion na bojisko, hráč 1 ho následne zoberie

Tento test kontroluje, ako sa položka (Healing Potion) presunie z inventára hráča 2 na bojisko a následne ju zoberie hráč 1.

Funkcia `sp_drop_item()` vyhodí predmet a funkcia `sp_loot_item()` ho vezme hráč 1.

Očakávaný výstup:

V `Dropped_goods` sa objaví záznam, že položka je dostupná na bojisku.

Hráč 1 získa položku do svojho inventára.

Hráč 2 už nemá daný predmet vo svojom inventári.

Test #6: Hráč 1 sa pokúsi použiť item na ktorý nemá dostatok AP

Tento test overuje, či sa hráčovi zakáže použiť predmet, ak nemá dostatok akčných bodov (AP).

Funkcia `sp_use_item()` sa pokúsi použiť predmet.

Očakávaný výstup:

Ak hráč nemá dostatok AP, transakcia by mala byť zrušená a nezaznamená sa žiadny záznam v `Round_events`.

Test #7: Hráč 2 zabije hráča 1 Fireballom

Tento test overuje, či je hráč 1 zabitý pomocou kúzla Fireball od hráča 2.

Funkcia `sp_cast_spell()` je použitá na útok.

Očakávaný výstup:

HP hráča 1 sa zníži na 0.

Hráč 1 sa následne vyhodí z aktívneho boja.

Všetky itemy a spelly hráča 1 sa dajú do tabuľky `dropped_goods`

Hráč 2 sa zlepší (level up).

Test #8: Reset kola – nové kolo vytvorené, AP pre hráča 2 resetované

Tento test kontroluje, či sa po resetovaní kola vytvorí nové kolo a AP pre hráča 2 sa správne resetujú.

Funkcia `sp_reset_round()` vytvorí nové kolo.

Očakávaný výstup:

Nové kolo je pridané do tabuľky `Rounds`.

AP pre hráča 2 sú resetované na maximálnu hodnotu.

Test #9: Pokus hráča 2 zobrať príliš ťažký item pre jeho inventár

Tento test kontroluje, či sa hráčovi 2 zakáže zobrať predmet, ak presahuje váhový limit jeho inventára.

Funkcia `sp_loot_item()` sa pokúša zobrať predmet.

Očakávaný výstup:

Predmet sa nepodarí zobrať.

Hráč 2 nebude mať daný predmet vo svojom inventári.

Test #10: Pokus hráča 2 zaútočiť na hráča 3, ktorý nie je v aktívnom combate

Tento test overuje, či sa hráč 2 pokúsi zaútočiť na hráča 3, ktorý nie je prítomný v boji.

Funkcia `sp_cast_spell()` vykoná pokus o útok.

Očakávaný výstup:

Systém zakáže útok, pretože hráč 3 nie je súčasťou aktívneho boja.

V `Round_events` sa nezaznamená žiadny záznam.

Test #11: Pokus hráča 2 sa pridať do aktívneho kola, ale hráč 2 už je v aktívnom kole

Tento test overuje, či sa hráč 2 môže pridať do aktívneho kola, ak už je v boji.

Funkcia `sp_enter_combat()` sa použije na pridanie hráča.

Očakávaný výstup:

Systém zakáže pridať hráča 2, pretože už je súčasťou boja.

V `Round_events` sa nezaznamená žiadny záznam.

Test #12: Vytvorenie nového combatu, pridanie hráča 1 do combatu, kontrola či sa zregeneroval kým nebol v boji

Tento test overuje, či sa HP hráča 1 správne zregeneruje pri vstupe do boja, ak nebol aktívny.

Funkcie `sp_start_combat()` a `sp_enter_combat()` sú použité na začatie boja.

Očakávaný výstup:

HP hráča 1 by malo byť zvýšené v dôsledku regenerácie.

Test #13: Ukončenie combatu, premiestnenie hráča 2 preč z aktívneho combatu

Tento test overuje, či správne funguje ukončenie boja a odstránenie hráča 2 z boja.

Funkcia `sp_end_combat()` ukončí boj a vyhodí hráčov.

Očakávaný výstup:

Záznam v `Round_events` ukáže ukončenie kola.

V `Player_combat_log` bude hráč 2 označený ako LEFT.

Test Views:

Tento test kontroluje správnosť zobrazenia rôznych pohľadov (views).

Skontroluje sa obsah pre:

`v_combat_state`

`v_most_damage`

`v_spell_statistics`

`v_strongest_characters`

`v_combat_damage`

`v_character_full_stats`

Záver

Testy implementácie prebehli úspešne a všetky hlavné funkcie systému boli overené. Je však možné, že existujú niektoré edge cases, ktoré ešte neboli úplne pokryté, ale časové obmedzenia mi nedovolili tieto prípady podrobne skontrolovať. Avšak, to, čo bolo implementované, by malo byť dostatočné na zabezpečenie správnej funkcionality systému v jeho základnom nastavení.

Použité nástroje

Pre vytvorenie diagramov bolo použité **lucid.app**

Pre vylepšenie syntaxu tejto dokumentácii bolo použité **chatgpt.com**